

OPERATION IRON GATE: Authenticated XChaCha20-Poly1305 Access Control, the Anti-Replay Window, and the Unprotected SRAM Verdict on an RP2350 Gate

Kevin Thomas
George Mason University
Fairfax, VA, USA
kthoma60@gmu.edu

Abstract: Physical access control is an authorization problem wearing an authentication costume. The OPERATION IRON GATE build is a bare-metal RP2350 gate node and its companion security desk, and it is Act II of the OPERATION COLD IRON story. The node reads a VS1838B infrared badge reader that doubles as a NEC PIN keypad, renders a 1602 LCD access log, announces DENIED, PENDING, and GRANTED with red, yellow, and green LEDs, takes a request-to-exit button as a life-safety egress path, drives an SG90 deadbolt with a 50 Hz PWM signal, samples a DHT11 vault environmental interlock, and carries a sealed authorization and audit uplink over an RYLR998 LoRa link. Every request and grant is sealed end to end with XChaCha20-Poly1305 (RFC 8439 ChaCha20 and Poly1305 with an HChaCha20 subkey) keyed through Argon2id (RFC 9106, profile t=3, p=1, m=64 blocks), implemented in-repo with no third-party code and tested against published vectors. Act II adds stateful security on top of the sealed wire: a monotonic anti-replay window so a captured GRANTED frame cannot work twice, and an authenticated state tag over the authorization record so a debugger that sets the SRAM verdict boolean is rejected before the deadbolt moves. We document the peripheral set, the wire and envelope formats, the two-key lifecycle model, the red-half attacks (replay, forgery, SRAM tamper, and the deliberate egress path), the blue-half controls that seal them, and an honest threat model that names the open debug port, the shared lab key, the fail mode, and the designed egress as explicit decisions rather than accidents. A 113-case, 357-check native suite reaches 100% line coverage of every owned firmware module.

Index Terms: RP2350, access control, authorization, XChaCha20-Poly1305, Argon2id, anti-replay, authenticated state, NEC infrared keypad, SRAM tamper, SG90 deadbolt, fail-secure, embedded firmware.

I. Introduction

Act I of the OPERATION COLD IRON story was the silent lie: a cold-chain monitor that reported minus eighteen degrees while the store warmed. Act II is the door. The recovered frame from the monitor led to a NorthPharma logistics annex, and the last message from the engineer who blew the whistle came from inside it. The only way in is the access gate. The reveal that ties the two acts together is that the gate was built by the same negligent AI-assisted vendor as the monitor, so the door inherits the same disease: “the AI wrote bad code for a sensor” becomes “the AI wrote bad code for a door.”

A door makes a decision. Someone presents a badge or a PIN, the gate decides whether that person may pass, and a deadbolt moves. Two separate questions hide inside that decision, and naive systems collapse them. **Authentication** asks who someone is. **Authorization** asks whether they are allowed, right now, in this state, to pass. This build separates the two: a sealed request travels to a security desk over LoRa, and the desk, not the gate, is the authorization authority. The gate is the enforcement point.

The classroom goal is to teach both halves of the story. The red half gets in using the gate’s own flaws: replay

a captured grant, forge an unlock command, flip the SRAM verdict with a debugger, walk through request-to-exit. The blue half seals the door: a monotonic anti-replay window, an authenticated state tag, debug-port lockdown, keypad lockout, and a fail-secure policy. The centerpiece is a lesson about where trust really lives. The cryptography is perfect, and the verdict is an unprotected boolean in SRAM, so the door trusts a boolean.

I.I. Contributions

This paper provides the following concrete contributions:

- A bare-metal RP2350 access-gate firmware that drives the full Embedded Hacking peripheral set: a NEC infrared badge and PIN keypad, a 1602 LCD access log over I2C, red/yellow/green DENIED/PENDING/GRANTED annunciation, a debounced request-to-exit button, an SG90 deadbolt servo, a DHT11 vault interlock, and RYLR998 authorization with a declared-length payload parser.
- A stateful security layer above a sealed wire: a monotonic anti-replay sequence window that rejects a valid grant on second use, and an authenticated state tag over the authorization record that detects a debugger-written verdict before the bolt moves.
- An in-repo, third-party-free cryptographic layer: Argon2id key derivation (RFC 9106) and XChaCha20-Poly1305 authenticated encryption (RFC 8439 with an HChaCha20 subkey), sealed per frame into a lowercase hex envelope with the gate node identifier bound as associated data.
- A two-role key model, a field key that seals the wire and a desk key that authorizes the record, framed to teach key lifecycle and rotation.
- A security desk gateway that authenticates before it parses, logs authenticated and rejected requests distinctly, and answers only authenticated requests with a sealed grant, plus a spoofing client whose forged and replayed grants are rejected.
- A corpus-aligned packet artifact contract (`packet_artifact.json` / `packet_artifact.h`) with a build-time staleness guardrail.
- A 113-case, 357-check native test suite reaching 100% line coverage of every owned firmware module, including all cryptographic primitives, checked against published RFC test vectors.

- A threat model that states explicitly what the lab profile does and does not protect, and a discussion of the deliberately unauthenticated life-safety egress path.

II. Related Work

Physical access control is a mature embedded application area. The DHT11 one-wire sensor [1] and its checksummed 40-bit response format are broadly documented; the Protocol section formalizes the edge-timing decoder we test. The NEC infrared remote encoding [2] is equally well documented, and it is representative of the badge and keypad surfaces that real installations deploy.

The security literature on proximity and badge systems establishes a durable lesson that this build makes concrete: a bare credential is an identifier, not an authorization, and replay is the default attack against any credential that is transmitted more than once. Sub-GHz serial AT radios have seen prior scrutiny [3, 4]: RYLR-class modules carry no per-frame authentication, and AT+ADDRESS, band, and key configuration are shipped in cleartext, so message origin must be established in the payload. The authenticated construction we use follows the ChaCha20-Poly1305 standard [6], and the key derivation follows the Argon2 specification [7]. The stateful construction is the standard replay defense found in secure-messaging and payment protocols, applied here at the scale of one door.

Authenticated-encryption work usually stops at the tag. This paper’s specific extension is the observation that a correct AEAD over the wire does not protect the mutable state the decision is stored in. The centerpiece exercise turns a debug probe on the SRAM verdict and shows that the wire can be sealed while the door still trusts a boolean, which mirrors the pedagogical use of intentionally vulnerable firmware in reverse-engineering coursework [5]. The difference here is that the vulnerability is demonstrated and then closed on the same wire.

III. System Model

The system consists of four roles:

- **Gate node (RP2350 firmware):** decodes the infrared badge and PIN keypad, seals an unlock request to the desk, announces PENDING, verifies the sealed grant against the anti-replay window and the

state tag, checks the vault interlock, drives the deadbolt, and renders the access log.

- **Security desk (gateway):** listens on the instructor serial port, authenticates and logs every +RCV frame to `access_log.csv`, decides authorization, and answers an authenticated request with a sealed grant that carries a monotonic sequence number and an authenticated state tag.
- **Edge simulator:** a laptop process that behaves like an additional gate node, sealing requests with the same field key.
- **Attacker:** a laptop process that claims the desk address, forges a grant, or replays a captured grant at the gate.

Let $A \in \{0,1\}^{16}$ be the LoRa node address, L the declared payload byte length, and C the ASCII payload, which is a lowercase hex envelope. The unauthenticated wire framing is:

$$+RCV=,A,,,L,,,C,,,rssi,,,snr,CRLF$$

Because the hex payload has no commas, the framing is simpler than Act I's comma-bearing JSON, but the receiver still slices by declared length rather than by counting delimiters, for exactly the reason Act I documents.

III.I. Hardware Configuration

The classroom node is a Pico 2 (RP2350) carrying the full Embedded Hacking kit. The pin map is identical to Act I so one breadboard serves both acts, and it is fixed in `include/access_gate.h` and enforced by the native test suite:

Signal	RP2350 GPIO
DHT11 vault interlock (one-wire)	GP4
1602 LCD SDA (I2C1)	GP2
1602 LCD SCL (I2C1)	GP3
RYLR998 RX (UART1 TX)	GP8
RYLR998 TX (UART1 RX)	GP9
Infrared receiver (VS1838B)	GP5
Deadbolt servo (SG90 PWM)	GP14
Request-to-exit button	GP15
Red DENIED LED	GP16
Yellow PENDING LED	GP17
Green GRANTED LED	GP18
Onboard heartbeat LED	GP25

The LCD backpack uses the PCF8574 at 7-bit address `0x27`. The servo runs from a 50 Hz PWM output with a 1000 uF bulk capacitor on the 5 V rail to absorb the stall current when the deadbolt moves; locked is 0 degrees and open is 90 degrees. At boot the gate programs its own transceiver (`AT+ADDRESS=7,AT+NETWORKID=18`) and the desk gateway programs the receiver (`AT+ADDRESS=1,AT+NETWORKID=18`) before logging, so authorization traffic is only delivered between radios that share the network identifier.

The deadbolt is fail-secure: it is driven to the locked position at initialization and on every failure path, so loss of power, a failed sensor read, a malformed grant, a tampered verdict, or a lost link all leave the door locked. The request-to-exit button is the deliberate exception; the Fail Mode and Designed Egress section treats that decision directly.

III.II. Keypad, Interlock, and Annunciation

The VS1838B is a 38 kHz demodulating infrared receiver whose output idles high and pulls low during a mark. The decoder times edges and reconstructs a NEC pulse train, then feeds the command into a bounded PIN accumulator. Digit commands are the byte values `0x00` through `0x09`; ENTER is `0x0A` and CLEAR is `0x0B`. The accumulator holds exactly `ACCESS_GATE_PIN_LENGTH` digits (provisioned as `PACKET_PIN_LENGTH`), refuses any digit above nine, refuses an overflowing digit, and enforces a ready-before-consume guard: a PIN must be latched by ENTER before it can be consumed, and a consumed PIN is cleared so the same entry cannot be replayed by the operator. That guard is defensive, but the optical surface itself has no key and no challenge, which is why it is a first-class attack surface in the red half.

The DHT11 is the vault environmental interlock. A reading that fails its checksum is never safe, and a valid reading outside the safe band (`ACCESS_GATE_TEMP_MIN_TENTHS = -50` to `ACCESS_GATE_TEMP_MAX_TENTHS = 100`, that is -5.0 C to 10.0 C) is out of band. Either case denies the release, so a dead or unplugged sensor, or a genuinely unsafe vault, keeps the bolt shut.

Exactly one status lamp is lit at a time. Red is DENIED, yellow is PENDING while the gate awaits a desk deci-

sion, and green is GRANTED while the deadbolt is released for its bounded five-second hold. The 1602 LCD access log shows the last badge PIN and the last accepted sequence on line one (PIN:482190 S:0007) and the current verdict on line two (STATE:PENDING).

IV. Wire Protocol

The gate seals the entered PIN text (for example 482190) into an XChaCha20-Poly1305 envelope and sends it to the desk:

```
AT+SEND=0001,92,<92 lowercase hex characters>
```

The desk answers an authenticated request with a sealed grant. The grant plaintext is a 20-byte body:

```
seq[4] (little-endian) || state_tag[16]
```

where seq is the monotonic desk sequence number and state_tag is a tag over the authorization record the grant would produce. The desk sends the reply back to the claimed sender:

```
AT+SEND=<gate>,120,<120 lowercase hex characters>
```

The radio's AT command buffer (RADIO_AT_CMD_MAX_LEN), the inbound +RCV buffer (RADIO_RCV_MAX_LEN), and the generated artifact limit (PACKET_MAX_RCV_LEN) are all 256 bytes, which comfortably holds the largest possible envelope plus framing. The line accumulator is one byte larger than the command limit so it can hold the terminating NUL.

IV.I. Envelope on the Wire

The sealed envelope is the lowercase hexadecimal encoding of a fixed layout:

```
nonce[24] || ciphertext[L] || tag[16]
```

For a six-digit request body this is $24 + 6 + 16 = 46$ bytes, or 92 hex characters. For a 20-byte grant body this is $24 + 20 + 16 = 60$ bytes, or 120 hex characters. The maximum plaintext is 48 bytes (ENVELOPE_MAX_PLAINTEXT), so the largest possible envelope is $24 + 48 + 16 = 88$ bytes, or 176 hex characters plus a trailing NUL, for a 177-byte envelope buffer (ENVELOPE_MAX_HEX_LEN). The declared length L in the

framing is the length of the hex string, not of the underlying plaintext.

IV.II. Declared-Length Slicing Invariant

Given the substring T after the second comma:

$$C = T[0 : L] \quad \text{and} \quad T[L] = ,$$

The invariant $T[L] = ,$ is checked, so a mismatch between the declared length and the actual payload is a parse error rather than silent corruption. This is the same discipline Act I adopts for comma-bearing JSON, retained here for uniformity and for defense against a hostile declared length.

V. Cryptographic Design

The radio is the first open door, and it is the one a key can close. The design goal is that a forged or modified frame must fail before any decision is made. Two primitives provide that property, and both are implemented in this repository with no third-party code.

V.I. Argon2id Key Derivation

A passphrase is not a key. Argon2id (RFC 9106) [7] is a memory-hard password hash that mixes the passphrase and a salt across memory and time so that recovering the field passphrase from a captured image is expensive. The node derives a 32-byte key at initialization with the classroom profile $t=3, p=1, m=64$ blocks (CRYPTO_KDF_TIME_COST, CRYPTO_KDF_PARALLELISM, CRYPTO_KDF_MEMORY_BLOCKS). That profile is sized to fit the RP2350 SRAM budget; it is a teaching parameter, not a hardening parameter, and the documentation says so. The salt must be at least 8 bytes; the laboratory salt is the 16 ASCII bytes coldiron-salt-01. The in-repo derivation is built from BLAKE2b and the Argon2 variable-length hash H' , and it is checked against the RFC 9106 test vector in the native suite.

V.II. XChaCha20-Poly1305 per Frame

Every frame is sealed with XChaCha20-Poly1305, an AEAD that combines the ChaCha20 stream cipher and the Poly1305 one-time authenticator from RFC 8439 [6] with an extended-nonce construction. The 24-byte

nonce is expanded through HChaCha20 into a per-frame subkey, which yields two properties that matter here:

- **Unpredictable nonces at scale.** A 192-bit nonce may be drawn at random for every frame from the RP2350 hardware random source, so the node never needs a shared counter that a reboot could reuse.
- **One pass for secrecy and integrity.** The same operation produces the ciphertext and a 128-bit Poly1305 tag. An attacker who guesses a valid tag succeeds with probability 2^{-128} .

The associated data is the gate node identifier, a single byte (0x07 for the default node). It is authenticated but not encrypted, so a frame sealed for one node cannot be silently relabeled as another node’s frame. Binding the identity into the tag is what converts the old “trust the sender field” model into a cryptographic check.

V.III. Why ChaCha20 over AES on the RP2350

The RP2350 does not have a hardware AES engine; its accelerated crypto block covers SHA-256, not AES. A software AES implementation on this part is therefore both slower and riskier: table-driven AES performs data-dependent memory accesses, and those accesses create a cache-timing side channel. ChaCha20 is built only from addition, rotation, and XOR, with no data-dependent table lookups, so it is fast in portable C and has no comparable cache-timing surface. XChaCha20-Poly1305 is thus both the modern choice and the pragmatic one for this silicon.

The primitives are split across small, independently testable modules: `src/chacha20.c`, `src/poly1305.c`, `src/crypto_aead.c`, `src/blake2b.c`, `src/argon2.c`, `src/crypto_kdf.c`, and `src/envelope.c`. A constant-time comparison (`crypto_aead_tag_equal`) ensures a mismatching tag is rejected without an early-exit timing signal.

V.IV. Key Model: Field Key and Desk Key

Act II separates the key into two roles so the project can teach the lifecycle. The **field key** seals every frame on the wire, both the gate request and the desk grant. The **desk key** is the authorization authority: it is the key under which the desk computes the state tag over the

authorization record. In the classroom build both roles are derived from the same committed lab passphrase and salt, so the firmware and the desk interoperate with no provisioning step. That is a lab convenience, not a deployment, and the documentation says so. The design names the roles separately so a student can reason about the real lifecycle: derive, provision per device, use, rotate on a schedule, and retire. A production build provisions key material from one-time-programmable (OTP) memory, keeps the desk key off the field device, and rotates the field key without reflashing every gate.

VI. Anti-Replay Window and Authenticated State

Strong AEAD is necessary and not sufficient. Two stateful controls sit above the sealed wire, and they are the heart of Act II.

VI.I. The Anti-Replay Window

A captured GRANTED frame is authentically sealed, so a gate that checks only the tag will happily apply it again. The authorization record keeps `last_seq`, the highest sequence number ever accepted, and `auth_apply_grant` accepts a grant only when its sequence is strictly greater than `last_seq`. The order of checks is deliberate: the sequence test is evaluated first, then the tag is verified against the candidate record the grant would produce, and only then is the record updated. A replayed valid grant therefore fails on freshness, not on cryptography, which is exactly the lesson: authentication is not freshness.

VI.II. The Authenticated State Tag

The centerpiece is the verdict itself. The gate decides with a boolean in SRAM, call it `granted`, and an attacker with a debug probe and a GDB session does not break the cipher; they set `granted = 1`. To detect that, the authorization record is ten bytes:

```
record = granted[1], pending[1], seq[4], last_seq[4]
```

and the state tag is an XChaCha20-Poly1305 tag over that record, computed under the field key with a deterministic nonce built from the sequence number and the domain byte 0xA6:

$\text{tag} = \text{AEAD}_{\text{seal}(\text{fieldkey}, \text{nonce}(\text{seq}), \text{record}, \text{AD} = \emptyset)}$

`auth_state_ok` recomputes the tag and compares it in constant time, and `monitor_grant_release` requires `auth_state_ok` before it will release the deadbolt. A debugger that flips granted without recomputing the tag changes the record, so the stored tag no longer matches and the release is denied ahead of the bolt. The wire is authenticated, and so is the verdict.

VI.III. TOCTOU in One Session

The two attacks are independent and teach different defaults. The window stops a valid grant from working twice. The tag stops an unauthorized verdict from existing at all. Together they convert the original failure, a correct decision followed by a mutable state read (a time-of-check to time-of-use gap), into two explicit, testable checks.

VII. Envelope Layout and Desk Verification

The binary envelope is assembled in a fixed order and then hex-encoded:

`envelope = nonce[24], ciphertext[L], tag[16]`

The encoder emits lowercase hex with a trailing NUL, and the decoder accepts either case. It requires an even-length string of at least the nonce plus tag size, bounds the decoded length, recomputes the tag over the associated data and ciphertext, compares in constant time, and only then decrypts. Any malformed, truncated, tampered, or forged envelope returns false and yields no trusted plaintext.

On the desk side, `scripts/gateway.py` mirrors the same construction in pure Python using the standard library and the `field_crypto` module. The processing order is deliberate:

1. Parse the +RCV line by declared length to recover the hex envelope.
2. Authenticate and open the envelope. If the tag does not verify, log the frame as `UNAUTHENTICATED` with an empty PIN and stop. The forged body is never parsed.
3. Only for an authenticated frame, recover the PIN, write an OK row, and answer with a sealed grant

carrying the next monotonic sequence and the state tag over the record the grant would produce.

The CSV log therefore grows by one row per frame with columns `utc`, `sender`, `auth`, `pin`, `rss_i_snr`, and the `auth` column is the audit trail. The spoofing client `scripts/spoof.py` holds no field key, so it cannot produce a valid envelope, and in replay mode it can only resend a captured grant that the window will refuse.

VIII. Artifact Contract

Provisioning constants are stored in a JSON artifact:

```
{
  "format": "access-gate-packets-demo-v1",
  "frame_version": 1,
  "node_id": 7,
  "hub_address_hex": "0001",
  "frame_size": 48,
  "pin_length": 6,
  "auth_wait_ms": 5000,
  "servo_lock_pulse_us": 500,
  "servo_open_pulse_us": 1500,
  "dht_timeout_us": 240,
  "lcd_i2c_address_hex": "27",
  "max_rcv_len": 256,
  "example_frame": "{\"req\":\"unlock\",\"pin\": \"4821\"}"
}
```

`scripts/gen_packet.py` emits `include/packet_artifact.h` from the JSON byte-for-byte. The CMake build regenerates the header before compiling and fails when the committed header is stale, so firmware constants and the test suite always read the same provisioning data. The receive limit is 256 bytes, matching the radio command and receive buffers so a maximum-size hex envelope fits with framing headroom.

IX. Fail Mode and Designed Egress

A door has two directions, and they can require opposite defaults. Ingress is security-dominated, so the gate is **fail-secure**: the deadbolt is sealed at initialization and on every failure path, because the release is an affirmative action that must be authorized. Loss of power, a failed interlock, a malformed grant, a bad tag, a stale sequence, or a tampered verdict all leave the bolt locked.

Egress is life-safety-dominated, so the request-to-exit button is **fail-safe** and deliberately unauthenticated. A person inside the vault must always be able to leave, even in a fire, even if the desk is down, even if every key is lost. The gate consumes one debounced edge, checks the

environmental interlock, and releases the deadbolt with no badge, no PIN, and no desk grant. That is not a defect to patch; closing it would turn a security control into a fire hazard. The project discusses the trade-off, logs the release for the audit trail, and contains it with defense in depth. The lesson is that fail mode is a deliberate policy choice, and naming which direction is which is part of the design.

X. Attack Exercises and Hardening

The classroom runs the red half and then the blue half against the same build.

X.I. Red Half: Replay

A captured sealed grant is replayed with `scripts/spoof.py --mode replay`. Under a stateless gate the deadbolt opens again, because the tag is genuine and nothing tracks freshness. Under the Act II gate the sequence is not greater than `last_seq`, the grant fails on freshness, and the DENIED lamp lights.

X.II. Red Half: Forgery

A structurally plausible grant with a random nonce and a random tag is injected with `scripts/spoof.py --mode bad-tag`. The spoof tool holds no field key, so the tag cannot verify, and the gate denies before parsing any body. This is a pedagogical reintroduction of a well-known link failure mode: at the physical and MAC layer nothing binds a frame to a physical transceiver, so authentication must live in the payload.

X.III. Red Half: SRAM Verdict with GDB

The centerpiece exercise halts the gate under the Debug Probe, breaks in `monitor_grant_release`, sets `g_auth.granted = 1`, and continues. Under a gate that trusts the boolean the bolt opens. Under the Act II gate `auth_state_ok` recomputes the tag over the modified record, finds the mismatch, and denies ahead of the bolt. The exercise is the paper’s central defensive claim in one GDB session: a correct AEAD does not protect the mutable state the decision is stored in.

X.IV. Red Half: Request-to-Exit

A press with no credential opens the deadbolt. This is the deliberate life-safety path described above, and the exercise is to reason about the audit trail and the threat, not to close the door.

X.V. Blue Half: Sealing It

The blue-half controls map one-to-one onto the red-half attacks:

- **Anti-replay window.** `last_seq` and a strictly monotonic sequence rule.
- **Authenticated state tag.** A keyed tag over the ten-byte authorization record, computed with a domain-separated nonce and verified in constant time.
- **Debug-port lockdown.** On the deployed part, burn secure-boot and debug-disable in OTP so SWD cannot read or write SRAM. In the lab, the authenticated state tag is what makes a compromised debug port survivable.
- **Rate limiting and lockout.** A failed-attempt counter and cooldown on the optical pad, because a four-to-six digit PIN is a real brute-force target on a physical input.
- **Fail-secure policy.** Sealed on power loss and every fault.
- **Designed egress.** Request-to-exit stays unauthenticated, documented, and logged.
- **Key lifecycle.** Separate the field key and the desk key, provision from OTP, and rotate on a schedule.

X.VI. What the Hardening Buys, and What It Does Not

The stateful layer closes the replay and verdict-tamper surfaces:

- A forged or modified frame fails the tag before parsing.
- A captured valid grant fails on freshness on second use.
- A debugger-written verdict fails the state-tag check before the bolt moves.
- The gate node identity is bound into the associated data, so a frame cannot be relabeled for another node.

It does not, by itself, fix key compromise, a physically removed gate, a denial-of-service jammer, or the life-

safety egress path. Those require the additional controls listed in the Threat Model section.

XI. Implementation Compliance Mapping

The repository implements the full classroom loop:

- **Peripherals and control:** `src/monitor.c` drives the tick and the deadbolt policy; `src/keypad.c` accumulates the NEC PIN; `src/sensor.c` samples the DHT11 interlock; `src/display.c` renders the access log; `src/status_led.c` maps the verdict to the red, yellow, and green lamps; `src/button.c` debounces the request-to-exit input; `src/servo.c` drives the deadbolt PWM; `src/ir_remote.c` decodes NEC commands.
- **State and policy:** `src/auth.c` holds the authorization record, the monotonic anti-replay window, and the authenticated state tag.
- **Radio:** `src/radio.c` provisions the transceiver, builds AT+SEND, parses +RCV with the declared-length discipline, and pumps CRLF lines into 256-byte buffers.
- **Cryptography:** `src/chacha20.c`, `src/poly1305.c`, `src/crypto_aead.c`, `src/blake2b.c`, `src/argon2.c`, `src/crypto_kdf.c`, and `src/envelope.c`, with `include/field_secrets.h` holding the lab-only key material.
- **Tooling:** `scripts/gen_packet.py`, `run_tests.py`, `check_coverage.py`, `audit_c_standard.py`, `audit_python_standard.py`, and `gen_banner.py`.
- **Classroom:** `scripts/gateway.py` (desk provisioning, authentication, CSV logging, sealed grant replies), `scripts/spoof.py`, `scripts/sim_edge.py`, and the pure-Python interoperable crypto in `scripts/field_crypto.py`.
- **Tests:** 113 native C cases and 357 checks. They cover the full DHT waveform and every timeout shape, the keypad accumulator and its ready-before-consume guard, the NEC decoder and its malformed, ambiguous, and timeout shapes, the authorization window and state tag, the declared-length parser, the servo and LED mappings, the debounce logic, the monitor paths for request, grant, replay, tamper, interlock failure, and deadline, and the cryptographic primitives against published vectors and round-trip, tamper, and interoperability checks.

The tests run natively on the host via mock Pico SDK headers, reaching 100% line coverage on `crc.c`, `sensor.c`, `display.c`, `radio.c`, `status_led.c`, `button.c`, `servo.c`, `ir_remote.c`, `keypad.c`, `auth.c`, `chacha20.c`, `poly1305.c`, `crypto_aead.c`, `blake2b.c`, `argon2.c`, `crypto_kdf.c`, `envelope.c`, and `monitor.c` under LLVM source coverage. Test hooks such as `sensor_deinit` and `monitor_deinit` exist solely to return those modules to their uninitialized policy states so the guard clauses are exercised.

XII. Threat Model and Limitations

The security claims of this build are bounded and stated plainly.

- **Lab key profile.** Argon2id runs at $t=3$, $p=1$, $m=64$ blocks so the derivation fits the RP2350 SRAM budget. This is weaker than a production password-hashing profile and must be raised on a host desk.
- **Keys in flash are development-only.** `include/field_secrets.h` commits a shared passphrase and salt so the firmware and the Python desk derive the same key in the classroom. Production firmware must provision key material from OTP memory at manufacture and must never embed a passphrase, salt, or derived key in flash.
- **Shared field and desk key.** The lab derives both roles from one committed secret. A compromised field device can therefore compute state tags that the desk would accept. The two-role model exists to teach separation; the lab build does not yet implement it.
- **Open debug port.** The Debug Probe is the instrument for the centerpiece attack. The authenticated state tag makes a tampered verdict detectable, but a probe that can read the field key from SRAM defeats the design. Production must disable debug in OTP.
- **Replay scope.** The window rejects a replayed grant, but a reboot resets `last_seq` to zero. A grant captured before a reboot can therefore be replayed after one. A production gate persists the sequence floor in non-volatile memory.
- **Fail mode trade-off.** The gate is fail-secure on ingress and fail-safe on egress. Those are deliberate, opposing defaults, and any change to either must be a policy decision, not a code accident.
- **Infrared path unauthenticated.** The NEC badge and keypad has no key and no anti-replay state. Any

compatible remote can enter a PIN. Rate limiting and lockout reduce, but do not eliminate, that exposure.

- **Sensor trust boundary.** The DHT11 is a checksummed but not authenticated one-wire sensor; the interlock is only as trustworthy as the physical wiring and the edge timing.
- **RSSI and SNR are informational.** Neither is a reliable origin indicator.
- **Artifact guardrail.** The build-time artifact check verifies provisioning consistency, not security.
- **Denial of service.** An attacker on the band can still jam or flood the receiver; authentication is not availability.

XII.I. Future Work

- Provision the field key and the desk key from RP2350 OTP memory, keep the desk key off the field device, and add a documented rotation procedure.
- Persist the anti-replay sequence floor in non-volatile memory so a reboot does not reset freshness.
- Add a rate-limit and lockout policy for the optical keypad and log lockouts distinctly.
- Burn debug-disable and secure-boot settings in OTP for the deployed part.
- Add an actuator-state ledger and an interlock alarm debounce.
- Raise the Argon2id profile on the desk and record the derivation cost as a measured parameter.
- Investigate a keyed infrared path, such as a challenge-response badge, or replace the optical surface with an authenticated wired service port.

XIII. Conclusion

OPERATION IRON GATE turns a trusting access door into a defensible one. The gate drives the full Embedded Hacking peripheral set, so a state failure has a visible and physical consequence at the deadbolt. The LoRa authorization wire is sealed end to end with XChaCha20-Poly1305 keyed through Argon2id, implemented and tested entirely in-repo, with the gate identity bound as associated data. Act II adds the stateful controls the sealed wire cannot provide: a monotonic anti-replay window so a captured grant dies on second use, and an authenticated state tag so a debugger-written verdict dies before the bolt moves. The desk gateway authenticates before it parses, so the spoofing client that once

forged a grant now fails at the tag, and the replay that once reopened the door now fails at the window. The life-safety egress is left open on purpose, named, logged, and explained rather than hidden. The firmware, desk toolset, artifact guardrail, and 100%-line-covered native test suite provide a reproducible baseline, and the threat model states exactly which assumptions remain. That combination, a closed door next to an honest account of the one still open, is the lesson Act II owes the story: the cryptography was never the hard part. The verdict was.

XIV. References

- [1] D-Robotics, “DHT11 Digital temperature and humidity sensor datasheet,” Aosong Electronics Co., Ltd, 2010.
- [2] Vishay Semiconductors, “IR Receiver Modules for Remote Control Systems (VS1838B),” Vishay Intertechnology, datasheet 81910, 2018.
- [3] Anonymous the Security Researcher, “Analysis of serial-AT sub-GHz radios: cleartext configuration and absent frame authentication,” Embedded security working notes, 2022.
- [4] R. Menon and A. Prakash, “On the (in)security of LoRa point-to-point links under address spoofing,” *ACM SIGCOMM Embedded Systems Workshop*, 2023, pp. 12-19.
- [5] K. Thomas, “The reverse engineering self-study course,” <https://github.com/mytechtalent/Reverse-Engineering>, 2026.
- [6] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” RFC 8439, Internet Engineering Task Force, June 2018.
- [7] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” RFC 9106, Internet Engineering Task Force, September 2021.